```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Created on Wed Nov 26 08:55:29 2025

@author: meppadilla
"""

import numpy as np
import matplotlib.pyplot as plt
from fpdf import FPDF
import datetime
import os

# ==========================================
# 0. CLASES Y FUNCIONES AUXILIARES
# ==========================================

class ReportePDF(FPDF):
    def header(self):
        self.set_font('Arial', 'B', 15)
        self.cell(0, 10, 'Reporte Tecnico: Simulacion de Aislamiento', 0, 1, 'C')
        self.set_font('Arial', 'I', 10)
        self.cell(0, 10, 'Cables Conectados a Bombas Sumergibles', 0, 1, 'C')
        self.ln(5)

    def footer(self):
        # Posición: a 2.0 cm del final
        self.set_y(-20)
        self.set_font('Arial', 'I', 8)

        # --- LINEA 1: FIRMA CON LINK ---
        self.set_text_color(100, 100, 100) # Gris oscuro
        self.cell(95, 5, 'Elaborado por: ', 0, 0, 'R')

        # Nombre con Link
        self.set_text_color(0, 0, 255) # Azul
        self.set_font('Arial', 'IU', 8)
        link_linkedin = 'https://www.linkedin.com/in/melvin-padilla-3425106'
        self.cell(95, 5, 'Ing. Melvin E. Padilla', 0, 1, 'L', link=link_linkedin)

        # --- LINEA 2: FECHA Y PAGINA ---
        self.set_text_color(128, 128, 128)
        self.set_font('Arial', 'I', 8)
        fecha = datetime.datetime.now().strftime("%d/%m/%Y %H:%M")
        self.cell(0, 5, f'Pagina {self.page_no()} - Generado el {fecha}', 0, 0, 'C')

    def chapter_title(self, label):
        self.set_font('Arial', 'B', 12)
        self.set_text_color(0, 0, 0)
        self.set_fill_color(200, 220, 255)
        self.cell(0, 6, label, 0, 1, 'L', 1)
        self.ln(4)

    def chapter_body(self, text):
        self.set_font('Arial', '', 11)
        self.multi_cell(0, 5, text)
        self.ln()

def solicitar_valor(mensaje, valor_defecto=None):
    txt_defecto = f" [Defecto: {valor_defecto}]" if valor_defecto is not None else ""
    while True:
        try:
            entrada = input(f"{mensaje}{txt_defecto}: ")
            if not entrada and valor_defecto is not None:
                return valor_defecto
            return float(entrada)
```

```
        except ValueError:
            print(">> Error: Ingrese un número válido.")

def paralelo(r_list):
    conductancia = sum([1/r for r in r_list])
    return 1 / conductancia

# ==========================================
# 1. INGRESO DE DATOS DE CAMPO
# ==========================================
print("\n" + "="*40)
print("   SISTEMA DE ANÁLISIS DE AISLAMIENTO")
print("="*40)

print("\n--- A. DATOS DE MEDICIÓN (MEGGER) ---")
r_medida_cable = solicitar_valor("1. Resistencia medida SOLO del cable (MΩ)", 2000)
l_medida_cable = solicitar_valor("2. Longitud de dicho cable (metros)", 200)

print("\n--- B. COMPONENTES DEL SISTEMA ---")
r_motor = solicitar_valor("3. Resistencia del MOTOR (MΩ)", 500)
r_empalme = solicitar_valor("4. Resistencia del EMPALME (MΩ)", 1000)

print("\n--- C. CONDICIONES AMBIENTALES ---")
t_ambiente = solicitar_valor("5. Temperatura Ambiente en superficie (°C)", 25)

# ==========================================
# 2. CÁLCULOS Y ESTIMACIONES
# ==========================================

# A. Cálculo de K
k_cable_calculada = r_medida_cable * (l_medida_cable / 1000.0)

# B. Evaluación
calidad_desc = ""
consejo = ""
if k_cable_calculada < 36:
    calidad_desc = "BAJA / CRITICA"
    consejo = "El cable presenta fugas importantes. No instalar sin revisar."
elif k_cable_calculada < 150:
    calidad_desc = "ESTANDAR (PVC)"
    consejo = "El cable tiene un comportamiento normal para PVC estandar."
else:
    calidad_desc = "EXCELENTE"
    consejo = "Cable en perfecto estado o de alta especificacion."

# C. Temperatura Fondo
delta_geotermico = (l_medida_cable / 100.0) * 3.0
delta_motor = 8.0
t_pozo_estimada = t_ambiente + delta_geotermico + delta_motor

# D. Factor de Corrección
delta_t = t_pozo_estimada - t_ambiente
factor_correccion = 0.5 ** (delta_t / 10.0)

# ==========================================
# 3. GENERACIÓN DE VECTORES
# ==========================================
prof_max = max(l_medida_cable, 200)
depths = np.linspace(1, prof_max, 200)
depths_km = depths / 1000.0

# --- Escenario 1: Ambiente ---
r_cable_amb = k_cable_calculada / depths_km
r_sistema_amb = [paralelo([rc, r_motor, r_empalme]) for rc in r_cable_amb]

# --- Escenario 2: Pozo ---
k_caliente = k_cable_calculada * factor_correccion
```

```python
r_motor_caliente = r_motor * factor_correccion
r_empalme_caliente = r_empalme * factor_correccion

r_cable_pozo = k_caliente / depths_km
r_sistema_pozo = [paralelo([rc, r_motor_caliente, r_empalme_caliente]) for rc in r_cable_pozo]

# Valores finales
val_final_amb = r_sistema_amb[-1]
val_final_pozo = r_sistema_pozo[-1]

# ==========================================
# 4. CREACIÓN DE LA GRÁFICA CON ETIQUETAS
# ==========================================
plt.figure(figsize=(11, 7))

# Curvas Ambiente (Azul)
plt.plot(depths, r_cable_amb, linestyle='--', color='blue', alpha=0.3, linewidth=1,
         label=f'Solo Cable @ {t_ambiente}°C')
plt.plot(depths, r_sistema_amb, linestyle='-', color='blue', linewidth=2.5,
         label=f'SISTEMA @ {t_ambiente}°C')

# Curvas Pozo (Rojo)
plt.plot(depths, r_cable_pozo, linestyle='--', color='red', alpha=0.3, linewidth=1,
         label=f'Solo Cable @ {t_pozo_estimada:.1f}°C')
plt.plot(depths, r_sistema_pozo, linestyle='-', color='red', linewidth=2.5,
         label=f'SISTEMA @ {t_pozo_estimada:.1f}°C')

# --- ETIQUETAS DE VALOR SOBRE LAS LÍNEAS (NUEVO) ---
# Etiqueta Azul (Sistema Frío)
plt.annotate(f"{val_final_amb:.1f} MΩ",
             xy=(prof_max, val_final_amb),
             xytext=(5, 5), textcoords='offset points',
             color='blue', fontweight='bold', fontsize=9)

# Etiqueta Roja (Sistema Caliente)
plt.annotate(f"{val_final_pozo:.1f} MΩ",
             xy=(prof_max, val_final_pozo),
             xytext=(5, -10), textcoords='offset points',
             color='red', fontweight='bold', fontsize=9)
# ---------------------------------------------------

plt.axhline(y=5, color='black', linestyle=':', linewidth=2, label='Lim. Operativo (5 MΩ)')

plt.yscale('log')
plt.grid(True, which="both", linestyle='-', alpha=0.2)
plt.title(f"Simulacion de Aislamiento: Ambiente vs Pozo\n(K: {k_cable_calculada:.1f} MΩ·km | Est
plt.xlabel("Profundidad (m)")
plt.ylabel("Resistencia (MΩ)")

# Aumentamos margen derecho para que quepan las etiquetas de texto
plt.xlim(0, prof_max * 1.15)

plt.legend(bbox_to_anchor=(1.02, 1), loc='upper left', borderaxespad=0.)
plt.tight_layout()

# Guardar imagen
nombre_imagen = "temp_grafica_aislamiento.png"
plt.savefig(nombre_imagen, dpi=100)
print(f"\n[INFO] Gráfica generada y guardada como '{nombre_imagen}'")

# ==========================================
# 5. GENERACIÓN DEL REPORTE PDF
# ==========================================
print("[INFO] Generando reporte PDF...")

try:
    pdf = ReportePDF()
```

```
        pdf.add_page()

        # --- Sección 1: Datos ---
        pdf.chapter_title("1. Datos de Entrada")
        datos_txt = (
            f"Resistencia Medida Cable: {r_medida_cable} MOhm\n"
            f"Longitud del Cable: {l_medida_cable} m\n"
            f"Resistencia Motor (Ref): {r_motor} MOhm\n"
            f"Resistencia Empalme: {r_empalme} MOhm\n"
            f"Temperatura Ambiente: {t_ambiente} C"
        )
        pdf.chapter_body(datos_txt)

        # --- Sección 2: Resultados ---
        pdf.chapter_title("2. Analisis Tecnico")
        resultados_txt = (
            f"Constante K Calculada: {k_cable_calculada:.2f} MOhm-km\n"
            f"Estado del Cable: {calidad_desc}\n"
            f"Temp. Estimada en Fondo: {t_pozo_estimada:.1f} C (Delta: +{delta_t:.1f} C)\n"
            f"Factor de Degradacion Termica: {factor_correccion:.4f}"
        )
        pdf.chapter_body(resultados_txt)

        # --- Sección 3: Predicción ---
        pdf.chapter_title(f"3. Prediccion a {prof_max} metros")
        prediccion_txt = (
            f"Resistencia Total en Frio (Superficie): {val_final_amb:.2f} MOhm\n"
            f"Resistencia Total en Caliente (Fondo): {val_final_pozo:.2f} MOhm\n\n"
            f"Evaluacion: {consejo}"
        )
        pdf.chapter_body(prediccion_txt)

        # --- Sección 4: Gráfica ---
        pdf.chapter_title("4. Curvas de Comportamiento")
        if os.path.exists(nombre_imagen):
            pdf.image(nombre_imagen, x=10, y=None, w=190)
        else:
            pdf.chapter_body("[ERROR: No se encontró la imagen de la gráfica]")

        # Guardar PDF
        nombre_pdf = "Reporte_Aislamiento.pdf"
        pdf.output(nombre_pdf)
        print(f"[EXITO] Reporte guardado como: {nombre_pdf}")

    except Exception as e:
        print(f"[ERROR] Fallo al crear el PDF: {e}")

    print("-" * 50)
    plt.show()
```